Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Дискретна математика

Лабораторна робота №3

«Графи. Способи представлення графів. Остовні дерева. Пошук найкоротших шляхів».

Виконав: студент
групи ІО-42
Куліков М. М.
Номер у списку групи: 18
Перевірив:
Пономаренко А. М.

Київ – 2024

Лабораторна робота №3

Тема: «Графи. Способи представлення графів. Остовні дерева. Пошук найкоротших шляхів».

Мета: Вивчення властивостей графів, способів їх представлення та основних алгоритмів на графах.

Загальне завдання:

1. Створити програму, яка реалізує один з алгоритмів на графах.

Теоретичні основи:

Останнім часом теорія графів стала простим, доступним і потужним засобом вирішення завдань, що відносяться до широкого кола проблем. Це проблеми проектування інтегральних схем і схем управління, дослідження автоматів, логічних ланцюгів, блок-схем програм, економіки та статистики, хімії та біології, теорії розкладів і дискретної оптимізації.

Граф G задають множиною точок або вершин x_1, x_2, ..., x_n (яку позначають через X) і множиною ліній або ребер a_1, a_2, ..., (яку позначають символом A), що з'єднують між собою всі або частину цих точок. Таким чином, граф G повністю задають (і позначають) парою (X, A).

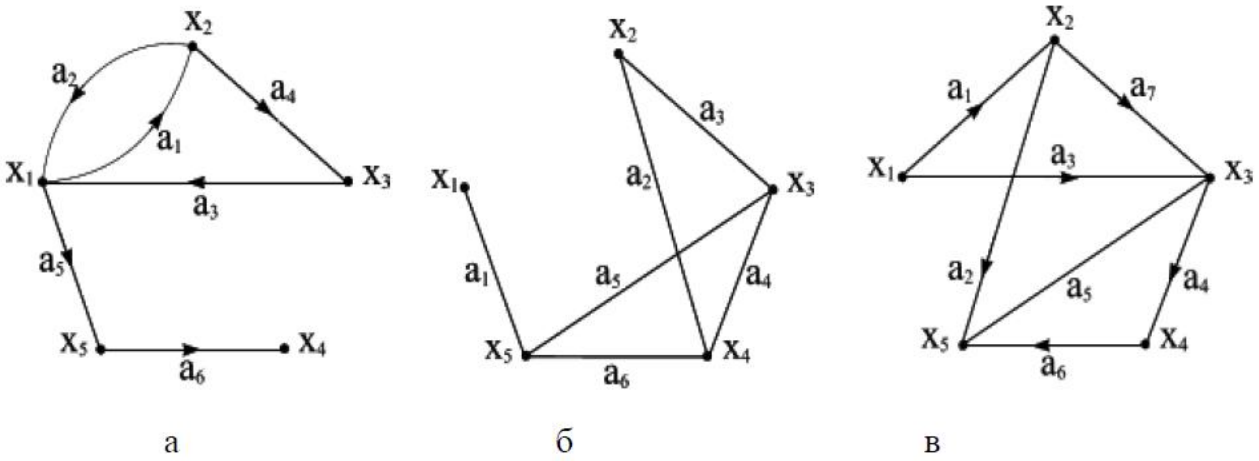

Рисунок 3.1 (а) – орієнтований граф; (б) – неорієнтований граф; (в) – змішаний граф

Якщо ребра з множини *A* орієнтовані, що зазвичай показується стрілкою, то їх називають дугами, і граф з такими ребрами називають орієнтованим графом (рис. 3.1 (а)). Якщо ребра не мають орієнтації, то граф називають неорієнтованим (рис. 3.1 (б)).

Якщо дугу позначають впорядкованою парою, що складається з початкової та кінцевої вершин (тобто двома кінцевими вершинами дуги), її напрямок передбачається заданим від першої вершини до другої. Так, наприклад, на рис. 3.1 (а) позначення (x_1, x_2) відноситься до дуги a_1, а (x_2, x_1) – до дуги a_2.

Інший, вживаний частіше, опис орієнтованого графа G полягає у задаванні множини вершин X і *відповідності* Г, яка показує, як між собою пов'язані вершини. Відповідність Г називають відображенням множини X в X, а граф в цьому випадку позначають парою $G = (X, Г)$.

Для графа на рис. 3.1 (а) маємо $Г(x_1) = \{x_2, x_5\}$, тобто вершини x_2 і x_5 є кінцевими вершинами дуг, у яких початковою вершиною є x_1.

$Г(x_2) = \{x_1, x_3\}$, $Г(x_3) = \{x_1\}$, $Г(x_4) = \emptyset$ - порожня множина, $Г(x_5) = \{x_4\}$

У разі неорієнтованого графа або графа, що містить і дуги, і неорієнтовані ребра (див., наприклад, графи, зображені на рис. 3.1 (б) і рис. 3.1 (в)), передбачається, що відповідність Г задає такий еквівалентний орієнтований граф, який отримуємо з початкового графа заміною кожного неорієнтованого ребра двома протилежно спрямованими дугами, що з'єднують ті ж самі вершини. Так, наприклад, для графа, наведеного на рис. 3.1 (б), маємо

$Г(x_5) = \{x_1, x_3, x_4\}$, $Г(x_1) = \{x_5\}$ і ін.

Оскільки пряма відповідність або образ вершини $Г(xi)$ є множиною таких x_j є X, для яких в графі G існує дуга (x_i, x_j), то через $Г^{-1}(x_i)$ природно позначити множину вершин x_k, для яких в G існує дуга $(x_к, x_j)$. Таку відповідність прийнято називати *зворотною відповідністю* або *прообразом* вершини. Для графа, зображеного на рис. 3.1(а), маємо

$Г^{-1}(x_1)=\{x_2, x_3\}$, $Г^{-1}(x_2)=\{x_1\}$ і т. д.

Цілком очевидно, що для неорієнтованого графа $Г^{-1}(x_i) = Г(x_i)$ для всіх x_i є X. Коли відображення Г діє не на одну вершину, а на множину вершин $X_q=\{x_1, x_2, ..., x_q\}$, то під $Г(X_q)$ розуміють об'єднання $Г(x_1) \cup Г(x2) \cup ... \cup Г(x_q)$, тобто $Г(X_q)$ є множиною таких вершин x_j є X, що для кожної з них існує дуга (x_i, x_j) в G, де x_i є X_q. Для графа, наведеного на рис. 3.1(а),

$Г(\{x_2, x_5\})=\{x_1, x_3, x_4\}$ і $Г(\{x_1, x_3\})=\{x_2, x_5, x_1\}$.

Відображення $Г(Г(x_i))$ записують як $Г^2(x_i)$. Аналогічно "потрійне" відображення $Г(Г(Г(x_i)))$ записують як $Г^3(x_i)$ і т. д. Для графа, показаного на рис. 3.1(а), маємо:

$\Gamma^2(x_1)=\Gamma(\Gamma(x_1))=\Gamma(\{x_2, x_5\})=\{x_1, x_3, x_4\}$;

$\Gamma^3(x_1)=\Gamma(\Gamma^2(x_1))=\Gamma(\{x_1, x_3, x_4\})=\{x_2, x_5, x_1\}$ і т. д

Аналогічно потрібно розуміти позначення $\Gamma^{-2}(x_i)$, $\Gamma^{-3}(x_i)$ і т. д.

Дуги $a=(x_i, x_j)$, $x_i \leftrightarrow x_j$, що мають спільні кінцеві вершини, називають суміжними. Дві вершини x_i і x_j називають суміжними, якщо яка-небудь з двох дуг (x_i, x_j) і (x_j, x_i) або обидві одночасно присутні в графі. Так, наприклад, на рис. 3.2 дуги a_1, a_{10}, a_3 і a_6, як і вершини x_5 і x_3, є суміжними, у той час як дуги a_1 і a_5 або вершини x_1 і x_4 не є суміжними.

Число дуг, які мають вершину *xi* своєю початковою вершиною, називають **напівстепенем виходу** вершини *xi*,, і, аналогічно, число дуг, які мають *xi* своєю кінцевою вершиною, називають напівстепенем входу вершини *xi*.

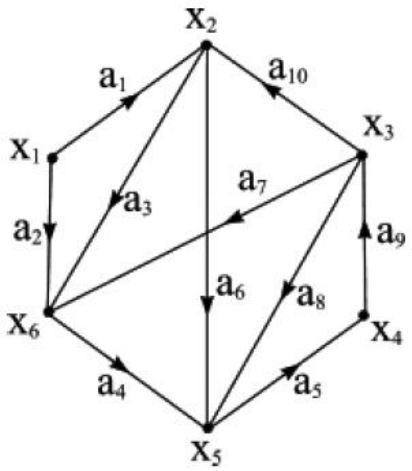

Рисунок 3.2 – граф

Таким чином, на рис. 3.2 напівстепінь виходу вершини x_3, позначена через $deg^+(x_3)$, дорівнює $|\Gamma(x_3)|=3$, і напівстепінь входу вершини x_3, позначена через $deg^-(x_3)$, дорівнює $|\Gamma^{-1}(x_3)|=1$.

Очевидно, що сума напівстепенів входу всіх вершин графа, а також сума напівстепенів виходу всіх вершин дорівнюють загальному числу дуг графа *G*, тобто

$$\sum_{i=1}^{n} deg^+(x_i) = \sum_{i=1}^{n} deg^-(x_i) = m, \quad (3.1)$$

де *n* – число вершин і *m* – число дуг графа *G*. Для неорієнтованого графа $G=(X, \Gamma)$ степінь вершини *xi* визначають аналогічно – за допомогою співвідношення $deg(x_i) =|\Gamma(x_i)|=|\Gamma^{-1}(x_i)|$.

Петлею називають дугу, початкова та кінцева вершини якої збігаються.

Одним з найбільш важливих понять теорії графів є дерево. *Неорієнтованим деревом* називають зв'язний граф, що не має циклів.

Суграфом графа G є підграф G_p, що містить всі вершини початкового графа. Якщо $G = (X, A)$ – неорієнтований граф з n вершинами, то зв'язний суграф Gp, який не має циклів, називають *остовним деревом (остовом) графа G*. Для остовного дерева справедливе співвідношення:

$$G_p=(X_p, A_p) \subseteq G, \text{ де } X_p = X, A_p \subseteq A \quad (3.2)$$

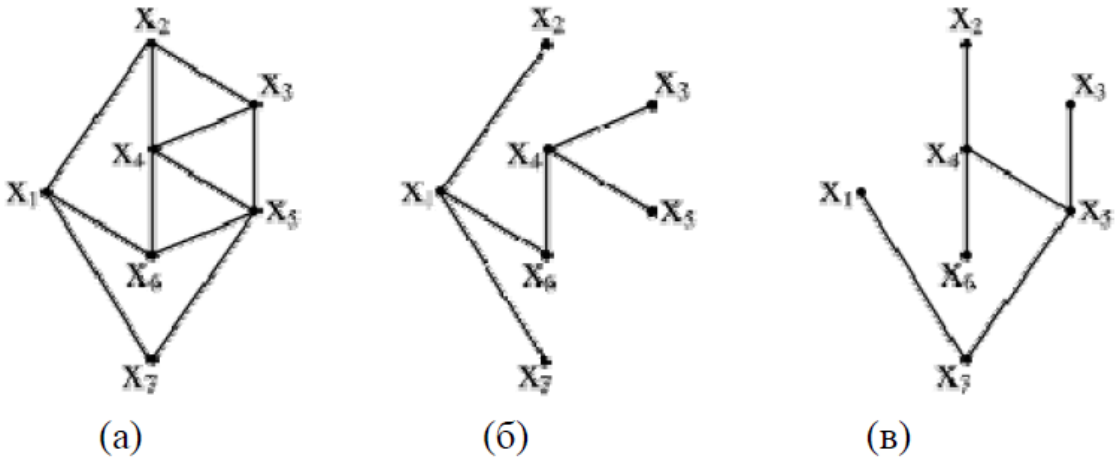

Рисунок 3.3 – представлення графа у вигляді остовних дерев

Легко довести, що остовне дерево має наступні властивості:

1) Остовне дерево графа з n вершинами має $n-1$ ребро ($|X_p|=|A_p|-1$);

2) Існує єдиний шлях, що з'єднує будь-які дві вершини остова графа: $\forall\, x_i,x_j \in X_p\ (i \neq j) \rightarrow \exists!\ \mu(x_i, x_j)$.

Наприклад, якщо G – граф, показаний на рис. 3.3(а), то графи на рис. 3.3(б, в) є остовами графа G. Зі сформульованих вище визначень випливає, що остов графа G можна розглядати як мінімальний зв'язний остовний підграф графа G.

Теорема Келі. На графі з n вершинами можна побудувати n^{n-2} остови дерев.

Орієнтоване дерево є орієнтованим графом без циклів, в якому напівстепінь входу кожної вершини, за винятком однієї (вершини r), дорівнює одиниці, а напівстепінь входу вершини r (названої коренем цього дерева) дорівнює нулю.

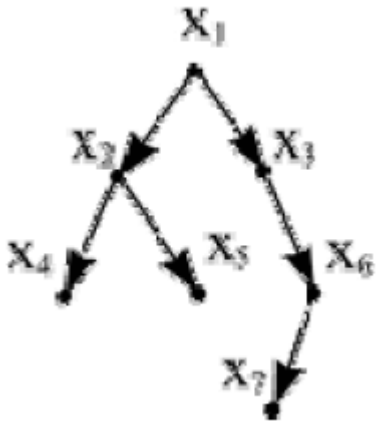

Рисунок 3.4 – орієнтоване дерево з коренем у вершині x_1

На рис. 3.4 показаний граф, який є орієнтованим деревом з коренем у вершині x_1. З наведеного визначення випливає, що орієнтоване дерево з n вершинами має $n-1$ дуг і є зв'язним.

Неорієнтоване дерево можна перетворити в орієнтоване: треба взяти його довільну вершину як корінь і ребрам приписати таку орієнтацію, щоб кожна вершина з'єднувалася з коренем простим ланцюгом.

"Генеалогічне дерево", в якому вершини відповідають особам чоловічої статі, а дуги орієнтовані від батьків до дітей, є добре відомим прикладом орієнтованого дерева. Корінь в цьому дереві відповідає "засновнику" роду (особі, народженій раніше за інших).

Шляхом (або *орієнтованим маршрутом*) орієнтованого графа називають послідовність дуг, в якій кінцева вершина будь-якої дуги, відмінної від останньої, є початковою вершиною наступної. Так, на рис. 3.6 послідовності дуг $\mu_1 = \{a_6, a_5, a_9, a_8, a_4\}$, $\mu_2 = \{a_1, a_6, a_5, a_9\}$, $\mu_3 = \{a_1, a_6, a_5, a_9, a_{10}, a_6, a_4\}$ є шляхами.

Орієнтованим ланцюгом називають такий шлях, в якому кожна дуга використовується не більше одного разу. Так, наприклад, наведені вище шляхи μ_1 і μ_2 є орієнтованими ланцюгами, а шлях μ_3 не є таким, оскільки дуга a_6 в ньому використовується двічі. Маршрут є неорієнтованим "двійником" шляху, і це поняття розглядається в тих випадках, коли можна знехтувати спрямованістю дуг у графі.

Таким чином, маршрут є послідовністю ребер $a_1, a_2, ..., a_q$, в якій кожне ребро a_i, за винятком, можливо, першого і останнього ребра, пов'язане з ребрами a_{i-1} і a_{i+1} своїми двома кінцевими вершинами.

Послідовності дуг на рис. 3.6 $\mu_4=\{a_2, a_4, a_8, a_{10}\}$, $\mu_5=\{a_2, a_7, a_8, a_4, a_3\}$ і

μ_6={a_{10}, a_7, a_4, a_8, a_7, a_2} є маршрутами.

Контуром (*простим ланцюгом*) називають такий шлях (маршрут), в якому кожна вершина використовується не більше одного разу. Наприклад, шлях μ_2 є контуром, а шляхи μ_1 і μ_3 – ні. Очевидно, що контур є також ланцюгом, але зворотне твердження невірне. Наприклад, шлях μ_1 є ланцюгом, але не контуром, шлях μ_2 є ланцюгом і контуром, а шлях μ_3 не є ні ланцюгом, ні контуром.

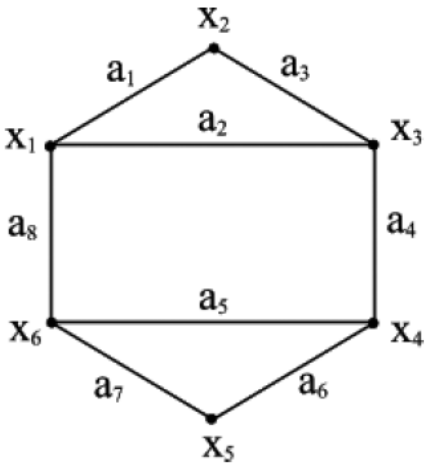

Рисунок 3.5

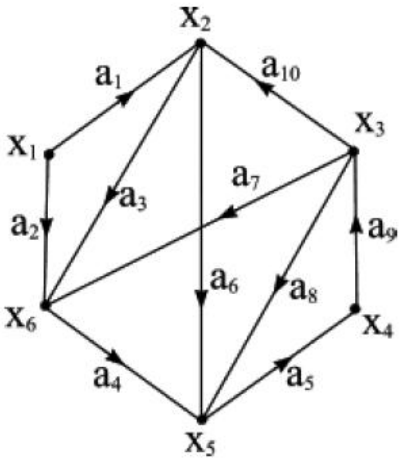

Рисунок 3.6

Аналогічно визначають простий ланцюг у неорієнтованих графах. Так, наприклад, маршрут μ_4 є простим ланцюгом, маршрут μ_5 – ланцюг, а маршрут μ_6 не є ланцюгом.

Шлях або маршрут можна зображати також послідовністю вершин.

Наприклад, шлях μ_1 можна представити так: μ_1 ={x_2, x_5, x_4, x_3, x_5, x_6} і таке представлення часто виявляється більш корисним у тих випадках, коли здійснюється пошук контурів або простих ланцюгів. Іноді дугам графа G співставляють (приписують) числа – дузі (x_i, x_j) ставлять у відповідність деяке число c_{ij}, назване **вагою**, або довжиною, або **вартістю** (ціною) дуги. Тоді граф G називають зваженим. Іноді ваги (числа *vi*) приписують вершинам x_i графа.

При розгляді шляху μ, представленого послідовністю дуг (a_1, a_2,...,a_q), за його вагу (або довжину, або вартість) приймають число $L(\mu)$, яке дорівнює сумі ваг всіх дуг, що входять до μ, тобто

$$L(\mu) = \sum_{(x_i,x_j)\in\mu} c_{ij} \quad (3.3)$$

Таким чином, коли слова "довжина", "вартість", "ціна" і "вага" застосовують до дуг, то вони еквівалентні за змістом, і в кожному конкретному випадку вибирають таке слово, яке краще відповідає змісту

завдання. **Довжиною** (або потужністю) шляху називають кількість (число) дуг, що входять до нього.

Обхід у глибину (англ. Depth-First Search, DFS) – один з основних рекурсивних методів обходу вершин графа. Загальна ідея алгоритму полягає в наступному: для кожної не відвіданої вершини необхідно знайти всі не відвідані суміжні вершини і повторити пошук для них. Таким чином, стратегія обходу в глибину наступна: йти «вглиб», поки це можливо, коли є не пройдені «вихідні» (ті, що виходять з вершини) ребра, і повертатися, щоб шукати інший шлях, коли таких ребер немає. Вперше «відвідавши» вершину v графа, відзначаємо цю подію, проставляючи в масиві used пройдених вершин значення true, used[v] <= true. В результаті обходу в глибину знаходяться перші лексикографічні шляхи в графі з відвіданої вершини в усі досяжні з неї.

Початковий граф може бути як неорієнтованим, так і орієнтованим, суті методу це не змінює.

Дуги орієнтованого графа поділяють на категорії залежно від їх ролі в обході в глибину. Цю класифікацію застосовують при вирішенні різних завдань, що базуються на методі обходу. Наприклад, доведено, що орієнтований граф не має циклів тоді і тільки тоді, коли обхід в глибину не знаходить в ньому «зворотних» дуг.

Після роботи обходу в глибину на орієнтованому графі G побудований ліс G_π.

Дуги в цьому лісі класифікуються як наступні:

1. Дуги дерева – це безпосередньо дуги, з яких складається ліс G_π, побудований в результаті обходу.
2. Зворотні дуги – це все дуги (u, v), що з'єднують вершину u з її предком v в одному з дерев лісу G_π обходу в глибину. (Зустрічні дуги, що утворюють цикл довжини 2, можливі в орієнтованих графах, вважаються зворотними дугами також).
3. Прямі дуги – це дуги, що з'єднують вершину з її нащадком, але не входять до лісу G_π обходу в глибину.
4. Перехресні дуги – це всі інші дуги орієнтованого графа, які не попали в одну з перерахованих вище категорій. Такі дуги, наприклад, можуть з'єднувати дві вершини одного дерева обходу, якщо жодна з цих вершин не є предком іншої.

Неорієнтовані графи вимагають особливого розгляду, оскільки одне і те ж ребро (u, v) = (v, u) обробляється двічі, з двох кінців, і може потрапити в різні категорії. Будемо відносити його до тієї категорії, яка представлена раніше в вищенаведеній класифікації. За такої умови прямих і перехресних ребер в неорієнтованому графі не буде.

Загальний порядок виконання роботи:

A. Вибрати номер Z індивідуального варіанта відповідно до виразу:

I = NZK mod 10+1, де NZK – номер залікової книжки.

B. Програма обчислення варіанта:

```
G=4218
print("Номер залікової книжки:",G)
print("Мій варіант:",(G%10) +1)
```

C. За індивідуальним варіантом побудувати алгоритм пошуку найкоротших шляхів.

Індивідуальне завдання:

№ вар	Опис варіанту
9	1. Розглянути алгоритм пошуку шляхів у глибину та написати програму пошуку найкоротших шляхів у графі за цим алгоритмом. 2. Знайти найкоротший шлях між вершинами, номери яких задані викладачем. 3. Представити граф у графічній формі та виділити найкоротший шлях, сформований за алгоритмом пошуку в глибину.

Розрахунок варіанту:

Код програми:

```
G=4218
print("Номер залікової книжки: ",G)
print("Мій варіант:",(G%10) +1)
```

Результат виконання програми:

```
"D:\Documents\Education\KPI\2 Term\
Номер залікової книжки: 4218
Мій варіант: 9
```

Рисунок 4.1 – результат обрахунку варіанта

Код програми:

Вміст файлу LR3.py:

```
 import tkinter as tk
from tkinter import messagebox, filedialog
import networkx as nx
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import random
import varCheck as variant

class GraphApp:
    def __init__(self, master):
        self.master = master
        self.master.title("ЛР3")
        main_frame = tk.Frame(master)
        main_frame.pack(fill="both", expand=True)
        control_frame = tk.Frame(main_frame)
        control_frame.pack(side="left", padx=10, pady=10, anchor="n")
        tk.Label(control_frame, text=f"Куліков Максим Миколайович\nГрупа: ІО-
{variant.G}\nНомер у списку: {variant.N}\nВаріант: {variant.get_variant_number()}",
justify="left").pack(anchor="w", pady=(0, 10))
        tk.Label(control_frame, text="Кількість вершин:").pack(anchor="w")
        self.entry_nodes = tk.Entry(control_frame)
        self.entry_nodes.pack()
        tk.Label(control_frame, text="Кількість ребер:").pack(anchor="w")
        self.entry_edges = tk.Entry(control_frame)
        self.entry_edges.pack()
        tk.Button(control_frame, text="Згенерувати граф",
command=self.generate_random_graph).pack(anchor="center", pady=(5, 15))

        tk.Label(control_frame, text="Початкова вершина:").pack(anchor="w")
        self.entry_start = tk.Entry(control_frame)
        self.entry_start.pack()
        tk.Label(control_frame, text="Кінцева вершина:").pack(anchor="w")
        self.entry_end = tk.Entry(control_frame)
        self.entry_end.pack()
        tk.Button(control_frame, text="Знайти шлях (У глибину)",
command=self.find_path).pack(anchor="center", pady=5)
        tk.Button(control_frame, text="Завантажити граф з файлу",
command=self.load_graph_from_file).pack(anchor="center", pady=(0, 5))
        tk.Button(control_frame, text="Зберегти поточний граф у файл",
command=self.save_graph_to_file).pack(anchor="center", pady=(0, 15))

        graph_frame = tk.Frame(main_frame)
        graph_frame.pack(side="right", padx=10, pady=10, expand=True, fill="both")
        self.graph = nx.Graph()
        self.fig, self.ax = plt.subplots(figsize=(6, 6))
        self.canvas = FigureCanvasTkAgg(self.fig, master=graph_frame)
        self.canvas.get_tk_widget().pack(fill="both", expand=True)

    def generate_random_graph(self):
        self.graph.clear()
        try:
            n = int(self.entry_nodes.get())
            m = int(self.entry_edges.get())
            if n <= 0 or m <= 0:
```

```
            raise ValueError
        if m > n * (n - 1) // 2:
            messagebox.showerror("Помилка", "Забагато ребер для заданої кількості вершин.")
            return

        self.graph.add_nodes_from([str(i) for i in range(n)])
        edges = set()
        while len(edges) < m:
            u, v = random.sample(list(self.graph.nodes), 2)
            if (u, v) not in edges and (v, u) not in edges:
                self.graph.add_edge(u, v)
                edges.add((u, v))
        self.draw_graph()
    except ValueError:
        messagebox.showerror("Помилка", "Введіть коректні числа для вершин і ребер.")

  def load_graph_from_file(self):
    file_path = filedialog.askopenfilename(title="Відкрити файл з графом", filetypes=[("Текстові
файли", "*.txt"), ("Усі файли", "*")])
    if not file_path:
        return
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            data = f.read().strip()
        pairs = data.split('),')
        edges = []
        nodes_set = set()
        for pair in pairs:
            part = pair.replace('(', '').replace(')', '').strip()
            if not part:
                continue
            u, v = [x.strip() for x in part.split(',')]
            edges.append((u, v))
            nodes_set.update([u, v])
        if not edges:
            raise ValueError
        self.graph.clear()
        self.graph.add_nodes_from(sorted(nodes_set))
        for u, v in edges:
            self.graph.add_edge(u, v)
        self.draw_graph()
    except Exception:
        messagebox.showerror("Помилка", "Не вдалося зчитати граф з файлу. Перевірте
формат.")

  def save_graph_to_file(self):
    if not self.graph.edges:
        messagebox.showerror("Помилка", "Немає ребер для збереження.")
        return
    file_path = filedialog.asksaveasfilename(title="Зберегти граф у файл", defaultextension=".txt",
filetypes=[("Текстові файли", "*.txt"), ("Усі файли", "*")])
    if not file_path:
        return
    try:
        edges = list(self.graph.edges)
        text = ", ".join(f"({u}, {v})" for u, v in edges)
        with open(file_path, 'w', encoding='utf-8') as f:
            f.write(text)
        messagebox.showinfo("Успіх", "Граф збережено у файл.")
```

```
        except Exception:
            messagebox.showerror("Помилка", "Не вдалося зберегти граф.")

    def find_path(self):
        if not self.graph.nodes:
            messagebox.showerror("Помилка", "Спочатку побудуйте або завантажте граф.")
            return
        start = self.entry_start.get().strip()
        end = self.entry_end.get().strip()
        if start not in self.graph or end not in self.graph:
            messagebox.showerror("Помилка", "Вказані вершини відсутні в графі.")
            return
        path = self.dfs_shortest_path(start, end)
        if not path:
            messagebox.showinfo("Результат", "Шлях не знайдено.")
        else:
            messagebox.showinfo("Результат", f"Знайдено шлях: {' -> '.join(path)}")
            self.draw_graph(path)

    def dfs_shortest_path(self, start, end):
        used = {v: False for v in self.graph.nodes}
        shortest = []
        def dfs(v, target, curr):
            nonlocal shortest
            used[v] = True
            curr.append(v)
            if v == target and (not shortest or len(curr) < len(shortest)):
                shortest = curr[:]
            for nbr in self.graph.neighbors(v):
                if not used[nbr]: dfs(nbr, target, curr)
            curr.pop(); used[v] = False
        dfs(start, end, [])
        return shortest

    def draw_graph(self, highlight=None):
        self.ax.clear()
        pos = nx.spring_layout(self.graph, seed=42)
        nx.draw(self.graph, pos, ax=self.ax, with_labels=True, node_color="lightblue", node_size=600)
        if highlight:
            nx.draw_networkx_edges(self.graph, pos, edgelist=list(zip(highlight, highlight[1:])),
edge_color="red", width=3)
        self.canvas.draw()

if __name__ == "__main__":
    root = tk.Tk()
    app = GraphApp(root)
    root.mainloop()
```

Вміст файлу varCheck.py:

```
G = 42
N= 18
def get_variant_number():
    return (int(str(G) + str(N))%10)+1
def print_variant_number():
    print("Номер залікової книжки:",G)
    print("Мій варіант:",get_variant_number())
print_variant_number()
```

Результат виконання програми:

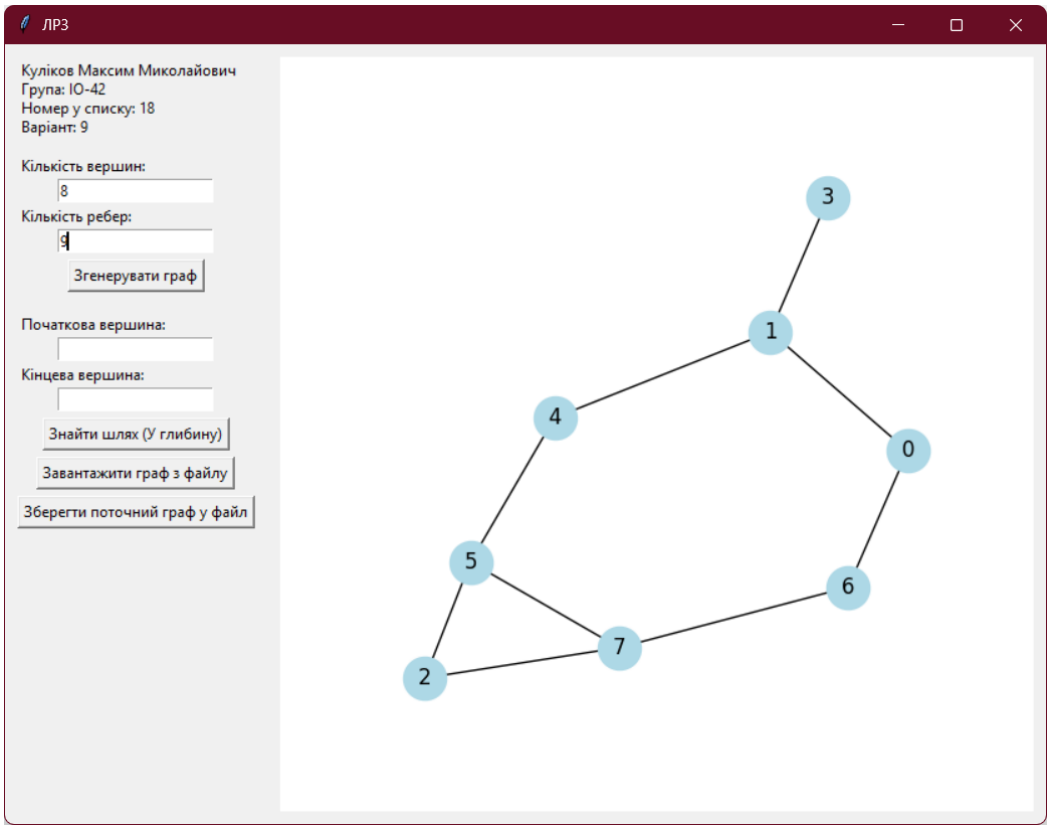

Рисунок 5.1 – результат виконання програми з утвореним графом

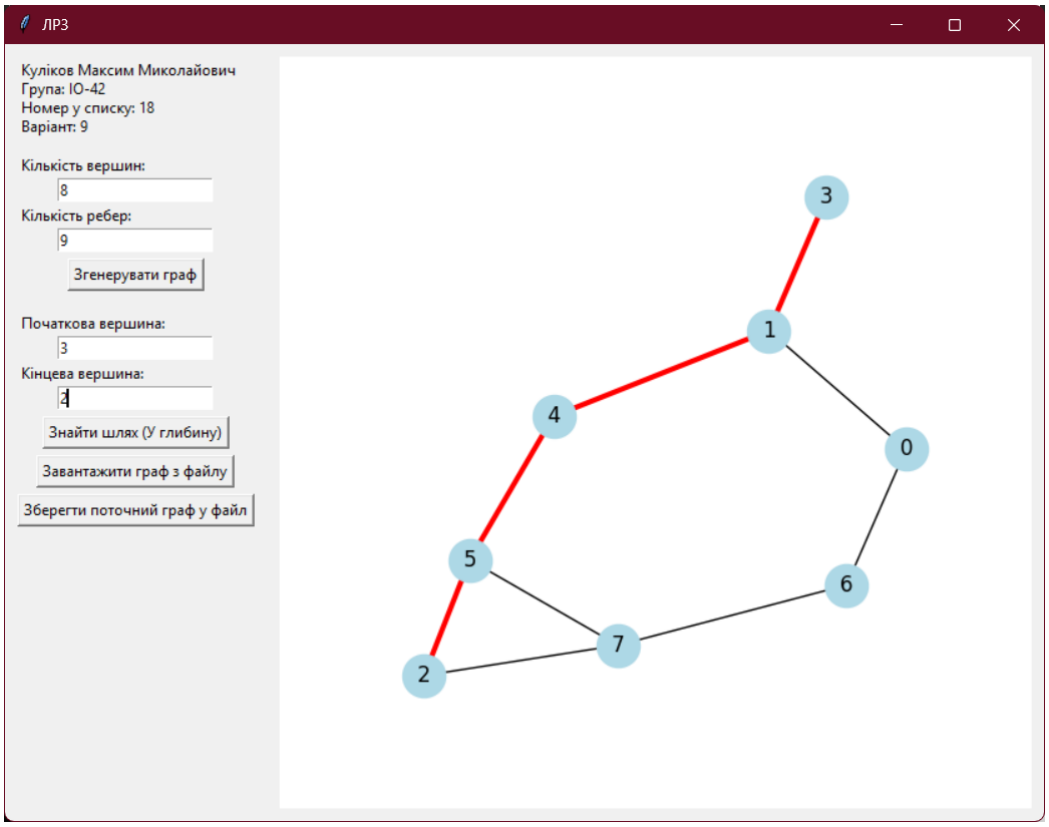

Рисунок 5.2 – результат виконання програми з знайденим найкоротшим шляхом

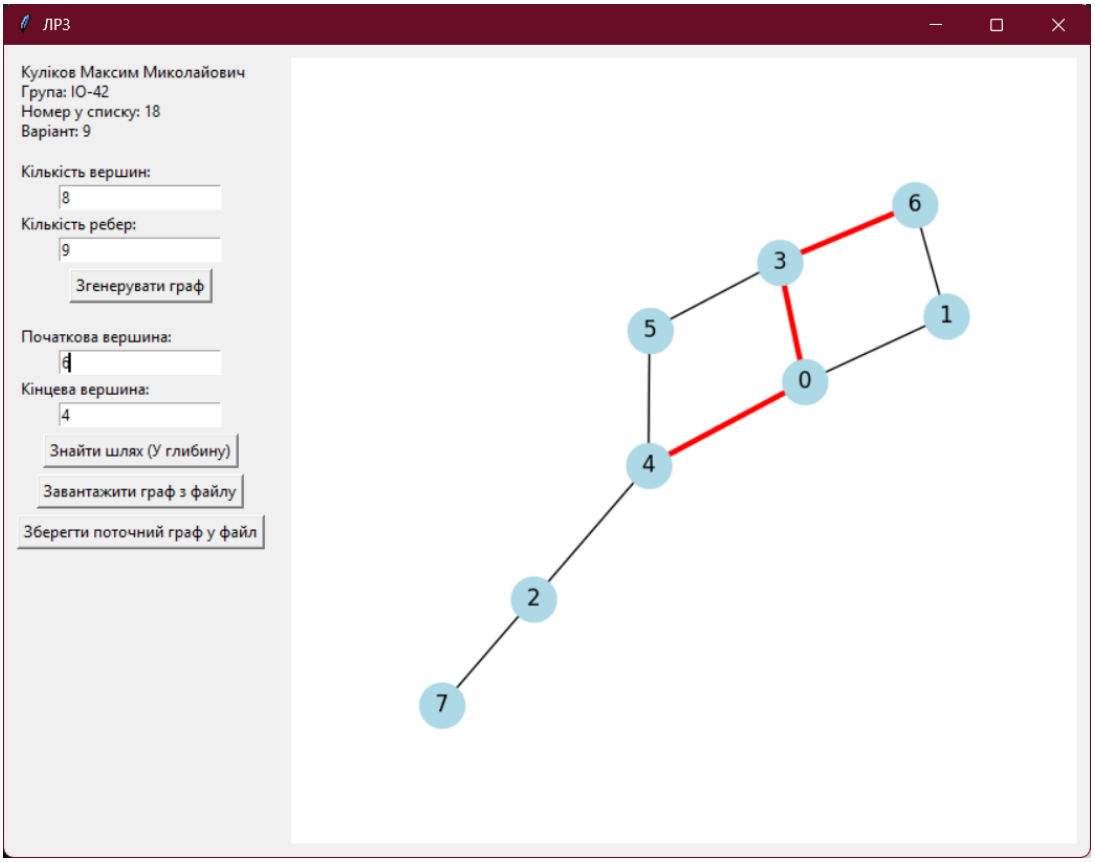

Рисунок 5.3 – результат виконання програми з іншим знайденим найкоротшим шляхом

Висновки:

Отже, під час виконання лабораторної роботи я розробив програму, яка шукає найкоротший шлях у графі за допомогою алгоритму пошуку в глибину. У процесі я закріпив знання про графи і принципи роботи алгоритму пошуку в глибину. Програма працює коректно та знаходить оптимальний маршрут. Проблем під час виконання не виникло. За потреби, програму можна вдосконалити, додавши підтримку зважених графів чи використати пошук у ширину, який у деяких випадках є ефективнішим.